Dokumentacja projektu końcowego

Implementacja i analiza porównawcza algorytmów Drzew Decyzyjnych oraz Lasów Losowych w zadaniach klasyfikacji

Dominik Bijoch, Michał Dobrowolski

27 stycznia 2026

Spis treści

1 Streszczenie założeń (projekt wstępny) 2

2 Opis wykorzystanych narzędzi i bibliotek 2

3 Opis zbiorów danych 2
3.1 Zbiór 1: Wisconsin Diagnostic Breast Cancer (WDBC) 2
3.2 Zbiór 2: Wine Quality (White and Red) 3

4 Opis funkcjonalny 3

5 Szczegółowy opis algorytmów i procesów przetwarzania danych 4
5.1 Przetwarzanie danych i dyskretyzacja 4
5.2 Algorytm Drzewa Decyzyjnego (DecisionTree) 4
5.3 Przycinanie metodą Reduced Error Pruning (REP) 4
5.4 Las Losowy (Random Forest) 5

6 Strojenie hiperparametrów lasu losowego (RandomForest) 5

7 Porównanie Drzew decyzyjnych i Lasu losowego 8
7.1 Zestawienie metryk jakości 9
7.2 Macierze pomyłek dataset: Breast Cancer (Labels: Benign, Malignant) 10
7.3 Macierze pomyłek dataset: Wine Quality Red (Labels: 3–8) 11
7.4 Macierze pomyłek dataset: Wine Quality White (Labels: 3–9) 12
7.5 Analiza i wnioski 13

8 Porównanie implementacji własnej z biblioteką scikit-learn 13
8.1 Wnioski 14

9 Podsumowanie 14

1 Streszczenie założeń (projekt wstępny)

Zadaniem projektu było zaimplementowanie i porównanie metod opartych na drzewach decyzyjnych i lasach losowych dla problemów klasyfikacyjnych. W szczególności:

- Implementacja klasycznego drzewa decyzyjnego wykorzystującego entropię i przyrost informacji (information gain).
- Implementacja przycinania drzewa (pruning) metodą Reduced Error Pruning (REP).
- Implementacja lasu losowego z bootstrapem i losowaniem podzbiorów cech (feature subsampling).
- Dyskretyzacja cech ciągłych przy użyciu metod binowania (KBinsDiscretizer) z heurystyką doboru liczby koszyków opartą o regułę Freedman–Diaconis.
- Porównanie modeli na trzech zbiorach danych: Breast Cancer (WDBC), Wine Quality (red i white).
- Porównanie własnej implementacji modeli z implementacją dostępną w bibliotece scikit-learn.

2 Opis wykorzystanych narzędzi i bibliotek

Projekt został zrealizowany w środowisku Python. Główne biblioteki i narzędzia:

- Python 3.11+
- `numpy` — operacje na tablicach i obliczenia numeryczne
- `pandas` — manipulacja danymi tabularycznymi oraz obsługa plików formatu CSV,
- `scikit-learn` (`sklearn`) — miary jakości (accuracy, precision, recall, f1), `train_test_split`, `KBinsDiscretizer`, `RandomForestClassifier` oraz `DecisionTreeClassifier` służące do porównania modeli.
- `matplotlib` — do szybkich wykresów podczas prototypowania

3 Opis zbiorów danych

Do badań wykorzystano dwa zbiory danych o odmiennej charakterystyce, pobrane z repozytorium UCI Machine Learning Repository. Atrybuty wejściowe w obu zbiorach mają charakter ciągły (rzeczywisty), co wymagało przeprowadzenia procesu dyskretyzacji przed etapem trenowania modeli drzew decyzyjnych. Dane zostały podzielone na zbiór treningowy i testowy w proporcji 70:30.

3.1 Zbiór 1: Wisconsin Diagnostic Breast Cancer (WDBC)

Ten zbiór danych dotyczy diagnostyki nowotworów piersi.

- **Typ zadania:** Klasyfikacja binarna (M = malignant/złośliwy, B = benign/łagodny).
- **Charakterystyka:** Zbiór zawiera 569 instancji. Z dokumentacji usunięto kolumnę identyfikacyjną (ID), która nie posiada wartości predykcyjnej.

- **Atrybuty:** Dane opisują cechy jąder komórkowych obecnych w obrazie aspiratu cienkoigłowego (FNA). Na podstawie każdego obrazu wyliczanych jest 10 podstawowych cech rzeczywistych, takich jak promień, tekstura, obwód, pole powierzchni czy gładkość. Dla każdej z tych cech obliczana jest średnia, błąd standardowy oraz wartość najgorsza (średnia trzech największych wartości), co sumarycznie daje 30 atrybutów wejściowych o charakterze ciągłym.

3.2 Zbiór 2: Wine Quality (White and Red)

Zbiór ten jest znacznie bardziej rozbudowany pod względem liczby instancji oraz złożoności atrybutów w porównaniu do zbioru medycznego.

- **Typ zadania:** Klasyfikacja wieloklasowa (ocena jakości).
- **Skala i cel:** Zbiór obejmuje dwa warianty portugalskiego wina „Vinho Verde": białe (4898 próbek) oraz czerwone (1599 próbek). Celem jest przewidywanie jakości wina w skali od 0 (bardzo złe) do 10 (doskonałe).
- **Atrybuty wejściowe:** Dane zawierają 11 ciągłych zmiennych wejściowych opartych na obiektywnych testach fizykochemicznych, takich jak pH, zawartość alkoholu, kwasowość, gęstość czy poziom cukru resztkowego.
- **Zmienna celu:** Wyjściowa ocena jakości opiera się na danych sensorycznych, stanowiących medianę z co najmniej trzech ocen wystawionych przez ekspertów.
- **Wyzwania:** Zbiór charakteryzuje się silnym niezbalansowaniem – najliczniej reprezentowane są wina o jakości średniej, podczas gdy klasy skrajne (bardzo słabe lub wybitne) są nieliczne. Ponadto, ze względu na korelacje między atrybutami fizykochemicznymi, zbiór ten jest odpowiedni do testowania metod selekcji cech.

4 Opis funkcjonalny

System składa się z następujących plików:

`preprocessing.py` Dyskretyzacja cech, podział na zbiory treningowe i testowe. Implementuje funkcje `get_bc_dataset`, `get_wr_dataset`, `get_ww_dataset` zwracające pary $(X_{train}, X_{test}, y_{train}, y_{test})$.

`DecisionTree.py` Implementacja klasy `DecisionTree` (bazowa klasa drzewa decyzyjnego), oraz klasy `DecisionTreeREP` (przycinanie REP).

`RandomForest.py` Implementacja klasy `RandomForest` jako agregacji wielu drzew (bootstrap + losowanie cech). Głosowanie większościowe zwraca etykietę końcową.

`compare_models.py` Skrypt wykonujący powtarzalne eksperymenty (wielokrotne losowe podziały) i porównujący miary jakości: accuracy, precision, recall, f1 oraz macierze pomyłek (confusion matrices) — średnia i suma.

`tune_hyperparameters.py` Skrypt do przeszukiwania siatki (grid search) hiperparametrów lasu losowego i zapisywania wyników.

`compare_with_sklearn.py` Skrypt do porównania własnych implementacji z gotowymi implementacjami drzew decyzyjnych dostępnymi w sklearn.

5 Szczegółowy opis algorytmów i procesów przetwarzania danych

5.1 Przetwarzanie danych i dyskretyzacja

Z uwagi na to, że implementacja drzew decyzyjnych opiera się na cechach dyskretnych, kluczowym elementem przetwarzania jest dyskretyzacja cech ciągłych. Wykorzystano do tego narzędzie `KBinsDiscretizer` (`scikit-learn`) z strategią kwantylową (`strategy='quantile'`). Liczba binów (`num_bins`) wyznaczana jest heurystycznie na podstawie reguły Freemana–Diaconisa, która minimalizuje błąd przybliżenia rozkładu:

$$\texttt{bin_width} = \frac{2 \cdot \mathrm{IQR}(X)}{n^{1/3}}$$

gdzie $\mathrm{IQR}(X)$ to rozstęp międzykwartylowy cechy X, a n to liczba próbek. W implementacji przyjęto ograniczenie $k \in [2, 15]$, co zapobiega nadmiernemu rozproszeniu danych przy małych zbiorach. Ostateczna liczba binów określana jest jako:

$$\texttt{num_bins} = \max\left(2, \min\left(\left\lfloor \frac{\max(X) - \min(X)}{\texttt{bin_width}} \right\rfloor, 15\right)\right)$$

5.2 Algorytm Drzewa Decyzyjnego (DecisionTree)

Implementacja w pliku `DecisionTree.py` buduje drzewo w sposób rekurencyjny. W każdym węźle algorytm dąży do maksymalizacji czystości podziału, wykorzystując miarę niepewności, jaką jest entropia:

$$H(Y) = -\sum_{i=1}^{C} p_i \log_2(p_i + \epsilon)$$

gdzie p_i to częstość występowania klasy i w całym zbiorze, a ϵ to stała zapobiegająca logarytmowaniu zera. Wybór optymalnej cechy do podziału odbywa się poprzez obliczenie przyrostu informacji (Information Gain):

$$IG(X, Y) = H(Y) - \sum_{v \in \mathrm{values}(X)} \frac{|Y_v|}{|Y|} H(Y_v)$$

Proces budowy węzła przebiega następująco:

1. Jeżeli wszystkie próbki w węźle należą do tej samej klasy lub nie pozostały cechy do podziału, tworzony jest liść z etykietą większościową.
2. Dla każdej dostępnej cechy obliczany jest $IG(X, Y)$. Wybierana jest cecha o najwyższej wartości przyrostu.
3. Dane są dzielone na podzbiory odpowiadające unikalnym wartościom wybranej cechy, a następnie dla każdego podzbioru procedura jest powtarzana.

5.3 Przycinanie metodą Reduced Error Pruning (REP)

Klasa `DecisionTreeREP` rozszerza podstawowe drzewo o mechanizm redukcji przeuczenia. Algorytm wykorzystuje niezależny zbiór walidacyjny (wydzielany z danych treningowych za pomocą `train_test_split`).

Procedura przycinania przebiega w kierunku od dołu do góry (post-pruning):

1. Dla każdego węzła wewnętrznego obliczana jest liczba błędów klasyfikacji na zbiorze walidacyjnym generowana przez całe poddrzewo zakorzenione w tym węźle.

2. Wynik ten porównuje się z liczbą błędów, które powstałyby po zastąpieniu całego poddrzewa pojedynczym liściem (z klasą dominującą).

3. Jeśli zastąpienie poddrzewa liściem nie zwiększa błędu walidacji, węzeł jest trwale przekształcany w liść.

5.4 Las Losowy (Random Forest)

Implementacja w `RandomForest.py` stanowi złożenie wielu drzew decyzyjnych. Algorytm łączy dwie techniki losowości w celu redukcji wariancji modelu:

- **Bagging (Bootstrap Aggregating):** Każde drzewo uczone jest na innym podzbiorze danych, losowanym ze zwracaniem. Wielkość podzbioru kontroluje parametr `sample_size`.
- **Feature Bagging:** W każdym drzewie (podczas wywołania metody `fit`) losowany jest podzbiór m cech (domyślnie $m = \sqrt{f}$, gdzie f to łączna liczba cech). Dzięki temu, że każde drzewo analizuje tylko wybrany zestaw cech, modele w zespole stają się bardziej różnorodne, co zapewnia ich pożądaną dekorelację.

Predykcja końcowa odbywa się na drodze głosowania większościowego:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \ldots, T_B(x)\}$$

gdzie $T_b(x)$ to predykcja b-tego drzewa, a B to liczba drzew w lesie (`n_trees`).

6 Strojenie hiperparametrów lasu losowego (RandomForest)

Strojenie polegało na przeszukaniu siatki kombinacji parametrów i ocenie średniej dokładności (accuracy) dla każdej kombinacji przy użyciu kilkukrotnego losowego podziału na zbiory trening/test. Przebadana siatka parametrów zawierała następujące wartości:

- `max_features`: {`'sqrt'`, `'log2'`}
- `n_trees`: {10, 50, 100, 200, 300}
- `sample_size`: {0.6, 0.8, 1.0}

Dla każdej kombinacji parametrów wykonano 10 powtórzeń z różnymi ziarnami (*seeds*), wyznaczając średnią oraz odchylenie standardowe dokładności. Wyniki dla poszczególnych zbiorów zestawiono w tab. 1–3. Badania ograniczono do `n_trees` = 300 ze względu na stabilizację wyników powyżej 200 drzew i brak istotnej poprawy jakości modelu uzasadniającej dalsze zwiększanie złożoności obliczeniowej.

Tabela 1: Wyniki strojenia hiperparametrów na zbiorze Breast Cancer

`n_trees`	`max_features`	`sample_size`	**Dokładność**	**Odchylenie std.**
10	log2	0.6	0.9228	0.0156
10	log2	0.8	0.9292	0.0170
10	log2	1.0	0.9275	0.0182
10	sqrt	0.6	0.9333	0.0198
10	sqrt	0.8	0.9310	0.0135
10	sqrt	1.0	0.9368	0.0125
50	log2	0.6	0.9322	0.0189
50	log2	0.8	0.9327	0.0214
50	log2	1.0	0.9363	0.0220
50	sqrt	0.6	0.9374	0.0172
50	sqrt	0.8	**0.9427**	0.0125
50	sqrt	1.0	**0.9427**	0.0133
100	log2	0.6	0.9292	0.0236
100	log2	0.8	0.9345	0.0192
100	log2	1.0	0.9363	0.0226
100	sqrt	0.6	0.9380	0.0148
100	sqrt	0.8	**0.9427**	0.0130
100	sqrt	1.0	0.9404	0.0113
200	log2	0.6	0.9333	0.0224
200	log2	0.8	0.9363	0.0212
200	log2	1.0	0.9357	0.0212
200	sqrt	0.6	0.9374	0.0183
200	sqrt	0.8	0.9380	0.0108
200	sqrt	1.0	0.9409	0.0106
300	sqrt	0.6	0.9386	0.0144
300	sqrt	0.8	0.9415	0.0152
300	sqrt	1.0	0.9421	0.0099

- Najwyższą dokładność (**0.9427**) odnotowano dla parametrów $n_trees \in \{50, 100\}$, $max_f =$ sqrt oraz $sample_s \in \{0.8, 1.0\}$.
- Wybór $max_f =$ sqrt systematycznie generował lepsze wyniki niż wariant `log2`, co wskazuje na korzyść z większej liczby cech branych pod uwagę przy podziale.
- Stabilizacja modelu następuje już przy 50 drzewach; dalsze zwiększanie `n_trees` do 300 nie poprawia istotnie dokładności, a jedynie minimalnie redukuje odchylenie standardowe (do poziomu **0.0099**).
- Modele wykorzystujące większą część zbioru treningowego (`sample_size` równe 0.8 lub 1.0) osiągały wyraźnie lepsze rezultaty niż te ograniczone do 60% danych.
- Jako optymalną konfigurację pod względem balansu między kosztem obliczeniowym a precyzjąm dla tego zbioru, wskazać można `n_trees` $= 50$, `max_features` $=$ sqrt, `sample_size` $=$ 0.8.

Tabela 2: Wyniki strojenia hiperparametrów dla zbioru: Wine Quality Red

`n_trees`	`max_features`	`sample_size`	**Dokładność**	**Odchylenie std.**
10	log2	0.6	0.6060	0.0209
10	log2	0.8	0.6150	0.0178
10	log2	1.0	0.6225	0.0157
10	sqrt	0.6	0.5852	0.0270
10	sqrt	0.8	0.5988	0.0326
10	sqrt	1.0	0.6083	0.0267
50	log2	0.6	0.6419	0.0210
50	log2	0.8	0.6390	0.0174
50	log2	1.0	0.6371	0.0137
50	sqrt	0.6	0.6394	0.0248
50	sqrt	0.8	0.6417	0.0174
50	sqrt	1.0	0.6487	0.0150
100	log2	0.6	0.6423	0.0194
100	log2	0.8	0.6415	0.0140
100	log2	1.0	0.6392	0.0154
100	sqrt	0.6	0.6483	0.0199
100	sqrt	0.8	0.6477	0.0220
100	sqrt	1.0	0.6512	0.0188
200	log2	0.6	0.6433	0.0206
200	log2	0.8	0.6460	0.0167
200	log2	1.0	0.6458	0.0165
200	sqrt	0.6	0.6565	0.0191
200	sqrt	0.8	0.6513	0.0210
200	sqrt	1.0	**0.6573**	0.0252
300	sqrt	0.6	0.6562	0.0199
300	sqrt	0.8	0.6550	0.0240
300	sqrt	1.0	0.6567	0.0235

- Najlepszy wynik dokładności (**0.6573**) osiągnięto dla konfiguracji $n_trees = 200$, $max_f =$ sqrt oraz $sample_s = 1.0$.
- Wzrost liczby drzew z 10 do 50 skutkuje gwałtownym skokiem dokładności (o ok. 3-4 punkty procentowe); dalsze zwiększanie `n_trees` powyżej 200 nie przynosi już istotnej poprawy wyników.
- Dla większych lasów ($n_trees \geq 50$) parametr $max_f =$ sqrt okazuje się skuteczniejszy niż `log2`, co sugeruje potrzebę większej liczby cech w węzłach przy bardziej złożonych modelach.
- Pełna próba treningowa ($sample_s = 1.0$) najczęściej prowadzi do najwyższych wyników dokładności, szczególnie w kombinacji z parametrem sqrt.
- Model wykazuje relatywnie wysokie odchylenie standardowe (powyżej **0.020** dla najlepszych wyników), co wskazuje na większą trudność klasyfikacji tego zbioru i mniejszą stabilność w porównaniu do zbioru Breast Cancer.

Tabela 3: Wyniki strojenia hiperparametrów dla zbioru: Wine Quality White

`n_trees`	`max_features`	`sample_size`	**Dokładność**	**Odchylenie std.**
10	sqrt	0.6	0.5729	0.0137
10	sqrt	0.8	0.5886	0.0132
10	sqrt	1.0	0.5988	0.0130
50	sqrt	0.6	0.6201	0.0102
50	sqrt	0.8	0.6255	0.0109
50	sqrt	1.0	0.6331	0.0103
100	sqrt	0.6	0.6239	0.0098
100	sqrt	0.8	0.6346	0.0105
100	sqrt	1.0	0.6389	0.0104
200	sqrt	0.6	0.6276	0.0085
200	sqrt	0.8	0.6382	0.0088
200	sqrt	1.0	**0.6418**	0.0082
300	sqrt	0.6	0.6280	0.0056
300	sqrt	0.8	0.6371	0.0096
300	sqrt	1.0	0.6403	0.0086

- Ze względu na to, że we wcześniejszych zbiorach parametr `max_features` = $\log_2$ systematycznie dawał słabsze wyniki, a duży rozmiar danych znacząco wydłużał czas obliczeń, zbiór *Wine Quality White* przetestowano wyłącznie dla parametru `max_features` = $\sqrt{n}$.
- Maksymalną dokładność (**0.6418**) uzyskano dla konfiguracji `n_trees` = 200, `max_features` = sqrt oraz `sample_size` = 1.0.
- Obserwuje się stały wzrost skuteczności modelu wraz ze zwiększaniem liczby drzew do poziomu 200; powyżej tej wartości (dla `n_trees` = 300) następuje stabilizacja wyniku.
- Parametr `sample_size` = 1.0 (pełna próba) we wszystkich przypadkach dla $n \geq 50$ okazał się najskuteczniejszy.
- Model wykazuje bardzo wysoką stabilność – odchylenie standardowe jest znacznie niższe niż w zbiorach Breast Cancer czy Wine Quality Red, osiągając minimum (**0.0056**) przy 300 drzewach.

Wybór konfiguracji do porównań końcowych Na podstawie wyników strojenia hiperparametrów do ostatecznych porównań wybrano model Random Forest z parametrami: `n_trees` = 250, `max_features` = 'sqrt' oraz `sample_size` = 1.0. Choć w przypadku zbioru *Breast Cancer* satysfakcjonujące wyniki osiągano już przy mniejszej liczbie drzew, zdecydowano się na jednakową konfigurację dla wszystkich trzech zbiorów. Dzięki temu wyniki są łatwiejsze do porównania, a model staje się bardziej uniwersalny.

7 Porównanie Drzew decyzyjnych i Lasu losowego

Poniżej przedstawiono zbiorcze zestawienie wyników dla 30 eksperymentów. Do analizy porównawczej modeli wykorzystano metryki:

- **Accuracy (dokładność):** procent wszystkich poprawnych przewidywań modelu w stosunku do całkowitej liczby próbek.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision (precyzja):** określa, jaki odsetek próbek zaklasyfikowanych jako pozytywne faktycznie należy do klasy pozytywnej.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (czułość, wykrywalność):** informuje, jak dobrze model odnajduje wszystkie rzeczywiste pozytywne przypadki.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score:** średnia harmoniczna Precision i Recall - używana, gdy zależy nam na równowadze między obiema metrykami.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **TP** - True Positive (prawidłowo wykryta klasa pozytywna)
- **TN** - True Negative (prawidłowo wykryta klasa negatywna)
- **FP** - False Positive (fałszywy alarm)
- **FN** - False Negative (przypadek pominięty)
- **Macierze Pomyłek** (Confusion Matrix)

7.1 Zestawienie metryk jakości

Tabela 4 prezentuje średnie wartości metryk dla wszystkich modeli i zbiorów danych.

Tabela 4: Porównanie średnich metryk jakości (mean ± std dla accuracy) dla własnej implementacji

Dataset	Model	Accuracy	Precision	Recall	F1
Breast Cancer	DecisionTree	0.9008 ± 0.0180	0.9019	0.9008	0.9006
	PrunedDecisionTree	0.9113 ± 0.0200	0.9126	0.9113	0.9104
	RandomForest	$\mathbf{0.9402 \pm 0.0189}$	0.9416	0.9402	0.9396
Wine Red	DecisionTree	0.6058 ± 0.0207	0.5981	0.6058	0.5998
	PrunedDecisionTree	0.5650 ± 0.0216	0.5402	0.5650	0.5453
	RandomForest	$\mathbf{0.6527 \pm 0.0228}$	0.6459	0.6527	0.6227
Wine White	DecisionTree	0.5839 ± 0.0107	0.5810	0.5839	0.5815
	PrunedDecisionTree	0.5246 ± 0.0112	0.5121	0.5246	0.5100
	RandomForest	$\mathbf{0.6432 \pm 0.0098}$	0.7053	0.6432	0.6180

7.2 Macierze pomyłek dataset: Breast Cancer (Labels: Benign, Malignant)

Model: Decision Tree

Średnia

	B	M
B	100.0	7.9
M	9.1	54.0

Suma

	B	M
B	3001	236
M	273	1620

Model: Pruned Decision Tree

Średnia

	B	M
B	102.8	5.1
M	10.6	52.5

Suma

	B	M
B	3083	154
M	319	1574

Model: Random Forest

Średnia

	B	M
B	105.2	2.7
M	7.1	56.0

Suma

	B	M
B	3155	82
M	213	1680

7.3 Macierze pomyłek dataset: Wine Quality Red (Labels: 3–8)

Model: Decision Tree

Średnia

	3	4	5	6	7	8
3	0.0	0.3	1.7	0.9	0.0	0.0
4	0.2	1.3	6.7	6.3	0.7	0.0
5	0.7	4.2	145.3	45.1	7.1	0.2
6	0.5	3.9	53.8	116.8	20.4	0.9
7	0.0	0.6	8.3	20.9	26.8	1.4
8	0.0	0.1	0.3	1.8	2.3	0.6

Suma

	3	4	5	6	7	8
3	0	10	51	28	0	0
4	6	38	202	189	22	0
5	21	125	4360	1352	212	6
6	16	118	1615	3503	611	28
7	0	18	248	626	803	41
8	0	2	8	54	68	19

Model: Pruned Decision Tree

Średnia

	3	4	5	6	7	8
3	0.0	0.1	1.9	1.0	0.0	0.0
4	0.2	0.6	7.5	6.6	0.3	0.0
5	0.4	2.6	145.6	49.6	4.2	0.1
6	0.2	2.7	68.4	110.5	14.3	0.3
7	0.0	0.4	7.2	35.7	14.2	0.3
8	0.0	0.1	0.2	3.1	1.3	0.3

Suma

	3	4	5	6	7	8
3	0	4	56	29	0	0
4	6	17	226	198	9	1
5	12	78	4369	1489	126	2
6	6	80	2051	3315	429	10
7	1	12	217	1071	427	8
8	0	4	7	93	39	8

Model: Random Forest

Średnia

	3	4	5	6	7	8
3	0.0	0.0	2.5	0.4	0.0	0.0
4	0.0	0.0	10.2	5.0	0.0	0.0
5	0.0	0.0	173.1	29.3	0.1	0.0
6	0.0	0.0	71.0	123.5	1.9	0.0
7	0.0	0.0	7.5	34.1	16.2	0.0
8	0.0	0.0	0.2	3.8	0.6	0.5

Suma

	3	4	5	6	7	8
3	0	0	76	13	0	0
4	0	0	306	151	0	0
5	0	0	5194	878	4	0
6	0	0	2131	3704	56	0
7	0	0	226	1023	487	0
8	0	0	5	113	19	14

7.4 Macierze pomyłek dataset: Wine Quality White (Labels: 3–9)

Model: Decision Tree

Średnia

	3	4	5	6	7	8	9
3	0.0	0.4	2.1	2.7	0.6	0.0	0.0
4	0.3	9.4	17.7	17.1	2.5	0.4	0.1
5	1.4	15.2	272.6	124.8	23.1	1.9	0.0
6	1.9	13.5	131.9	422.0	74.9	11.5	0.3
7	0.6	3.7	24.8	93.2	133.8	10.1	0.1
8	0.3	0.6	2.9	17.2	12.4	20.6	0.0
9	0.0	0.1	0.0	1.0	0.2	0.1	0.0

Suma

	3	4	5	6	7	8	9
3	0	12	62	80	19	1	0
4	8	283	530	512	75	13	2
5	42	456	8177	3743	692	57	1
6	57	406	3958	12661	2247	346	8
7	17	111	743	2796	4014	303	2
8	10	18	88	516	373	617	1
9	0	2	1	31	7	2	0

Model: Pruned Decision Tree

Średnia

	3	4	5	6	7	8	9
3	0.0	0.3	1.7	3.3	0.5	0.0	0.0
4	0.0	3.4	19.7	21.6	2.7	0.1	0.0
5	0.8	7.7	239.4	172.6	17.1	1.3	0.0
6	0.4	7.1	147.7	422.2	73.7	4.9	0.0
7	0.1	1.6	23.4	138.5	99.0	3.6	0.0
8	0.0	0.2	1.9	29.6	15.3	7.1	0.0
9	0.0	0.0	0.1	1.0	0.3	0.0	0.0

Suma

	3	4	5	6	7	8	9
3	1	8	50	99	16	0	0
4	0	102	591	647	80	3	0
5	23	231	7181	5179	514	40	0
6	13	212	4432	12666	2212	147	1
7	3	47	702	4156	2970	108	0
8	1	5	56	887	460	214	0
9	0	0	2	31	10	0	0

Model: Random Forest

Średnia

	3	4	5	6	7	8	9
3	0.0	0.0	2.1	3.7	0.0	0.0	0.0
4	0.0	4.4	18.3	24.6	0.1	0.0	0.0
5	0.0	0.1	265.4	172.9	0.6	0.0	0.0
6	0.0	0.0	80.5	569.0	6.6	0.0	0.0
7	0.0	0.0	5.3	169.2	91.6	0.1	0.0
8	0.0	0.0	0.1	35.6	3.4	15.1	0.0
9	0.0	0.0	0.0	1.4	0.0	0.0	0.0

Suma

	3	4	5	6	7	8	9
3	0	0	64	110	0	0	0
4	0	133	550	738	2	0	0
5	0	2	7962	5186	18	0	0
6	0	0	2415	17071	197	0	0
7	0	0	159	5075	2748	4	0
8	0	0	3	1067	101	452	0
9	0	0	0	43	0	0	0

7.5 Analiza i wnioski

Na podstawie przeprowadzonej serii 30 eksperymentów dla każdego ze zbiorów danych, sformułowano następujące wnioski:

- **Dominacja Lasu Losowego:** Model **Random Forest** konsekwentnie osiągał najwyższe wyniki we wszystkich mierzonych metrykach. Jego przewaga nad pojedynczym drzewem decyzyjnym (zazwyczaj o $4 - 8$ punktów procentowych) wynika z mechanizmu *ensemble learningu*. Agregacja wyników wielu drzew oraz wprowadzenie losowości na etapie wyboru cech i próbek skutecznie poprawia generalizację modelu.

- **Efektywność Przycinań (Pruning) i Złożoność Modelu:** Zastosowanie techniki *Reduced Error Pruning* (REP) ma kluczowy wpływ na strukturę modelu. Jak wykazano na przykładzie zbioru *Breast Cancer*, model po przycięciu staje się znacznie mniej rozbudowany, co pozytywnie wpływa na jego interpretowalność:

Model	Średnia liczba węzłów	Średnia liczba liści	Średnia głębokość
DecisionTree	109.7 ± 9.8	96.7 ± 8.5	3.0 ± 0.0
PrunedDecisionTree	28.4 ± 12.8	25.8 ± 11.3	2.1 ± 0.9

Tabela 5: Porównanie złożoności drzewa przed i po przycięciu dla zbioru Breast Cancer.

  Redukcja liczby węzłów o blisko 75% przy jednoczesnym zachowaniu (a w przypadku *Breast Cancer* nawet poprawieniu) wyników *Accuracy* świadczy o tym, że standardowe drzewa decyzyjne mają tendencję do tworzenia nadmiarowych gałęzi, które nie wnoszą istotnej informacji, a jedynie dopasowują się do szumu w danych.

- **Specyfika Zbiorów Danych:**
  - W zbiorze *Breast Cancer* przycinanie poprawiło wyniki (0.9113 vs 0.9008), co może sugerować występowanie minimalnego przeuczenia w pełnym modelu.
  - W zbiorach *Wine Quality* przycinanie pogorszyło wyniki. Wskazuje to na dużą złożoność problemu klasyfikacji jakości wina, gdzie uproszczony model nie jest w stanie wyznaczyć precyzyjnych granic decyzyjnych między zbliżonymi do siebie klasami (np. ocenami 5 i 6).

- **Analiza Błędów:** Macierze pomyłek ujawniają, że najwięcej błędów klasyfikacji występuje między klasami sąsiadującymi. W przypadku win, cechy fizykochemiczne próbek o jakości "5"i "6"są często niemal identyczne, co sprawia, że modele mają trudność z ich jednoznacznym rozróżnieniem. Las Losowy najlepiej radzi sobie z tym wyzwaniem dzięki agregacji decyzji z wielu drzew.

- **Interpretowalność vs Skuteczność:** Wybór modelu zależy od priorytetów zadania. Jeśli kluczowa jest precyzja, **Random Forest** jest bezkonkurencyjny. Jeśli jednak priorytetem jest większa przejrzystość decyzji i łatwość analizy reguł przez człowieka, **PrunedDecisionTree** oferuje najlepszy kompromis, drastycznie upraszczając strukturę drzewa przy zachowaniu akceptowalnej jakości predykcji.

8 Porównanie implementacji własnej z biblioteką scikit-learn

W celu walidacji poprawności działania zaimplementowanych algorytmów, przeprowadzono testy porównawcze z analogicznymi modelami z biblioteki `scikit-learn`. Wyniki uśrednione z 30 eksperymentów dla trzech zbiorów danych przedstawiono w tabeli 6, gdzie DT oznacza Decision

Tree, natomiast RF oznacza RandomForest. Niestety biblioteka scikit-learn nie oferuje implementacji przycinania metodą REP (Reduced Error Pruning). Dostępna jest jedynie metoda Cost Complexity Pruning (parametr `ccp_alpha`), co sprawia, że ich bezpośrednie porównanie jest nieuzasadnione, ponieważ są to dwa odmienne podejścia do post-pruningu

Tabela 6: Porównanie metryk jakości: implementacja własna vs scikit-learn

Dataset	Model	Accuracy (± std)	Precision	Recall	F1
Breast Cancer	DT (własny)	0.9008 (± 0.0180)	0.9019	0.9008	0.9006
	DT (sklearn)	**0.9365 (± 0.0152)**	0.9383	0.9365	0.9365
	RF (własny)	0.9402 (± 0.0183)	0.9415	0.9402	0.9396
	RF (sklearn)	**0.9577 (± 0.0141)**	0.9583	0.9577	0.9576
Wine Red	DT (własny)	**0.6058 (± 0.0207)**	0.5981	0.6058	0.5998
	DT (sklearn)	0.5877 (± 0.0209)	0.5928	0.5877	0.5891
	RF (własny)	0.6543 (± 0.0215)	0.6467	0.6543	0.6245
	RF (sklearn)	**0.6749 (± 0.0174)**	0.6501	0.6749	0.6579
Wine White	DT (własny)	0.5839 (± 0.0107)	0.5810	0.5839	0.5815
	DT (sklearn)	**0.5902 (± 0.0113)**	0.5923	0.5902	0.5906
	RF (własny)	0.6412 (± 0.0105)	0.7036	0.6412	0.6160
	RF (sklearn)	**0.6739 (± 0.0112)**	0.6828	0.6739	0.6623

8.1 Wnioski

- **Poprawność implementacji**: Wyniki uzyskiwane przez modele własne są zbliżone do wyników referencyjnych z biblioteki `scikit-learn`, co potwierdza poprawność implementacji własnej.
- **Optymalizacja obliczeniowa**: Mimo porównywalnej skuteczności predykcyjnej, modele `scikit-learn` wykazują znacznie wyższą optymalizację pod kątem wydajnościowym. Jest to szczególnie zauważalne przy predykcji lasu losowego (*RandomForest*) składającego się z 250 drzew na większych zbiorach danych, takich jak *Wine Quality*.

9 Podsumowanie

Zrealizowany projekt pozwolił na praktyczną weryfikację działania klasycznych algorytmów uczenia maszynowego poprzez ich własną implementację od podstaw. Przeprowadzone eksperymenty jednoznacznie wykazały przewagę Lasu Losowego (Random Forest) nad pojedynczymi drzewami decyzyjnymi. Dzięki zastosowaniu technik *baggingu* oraz losowania podprzestrzeni cech, model ten skutecznie redukuje wariancję, osiągając najwyższą stabilność i dokładność na wszystkich badanych zbiorach danych (średnio o 4–8 p.p. lepiej niż pojedyncze drzewo).

Analiza mechanizmu przycinania (Reduced Error Pruning) dostarczyła istotnych wniosków dotyczących kompromisu między dokładnością a interpretowalnością. Metoda ta pozwala na drastyczną redukcję złożoności modelu (zmniejszenie liczby węzłów nawet o 75%), co czyni drzewo czytelnym dla człowieka. Jednakże, w przypadku danych o wysokim poziomie szumu lub małej separowalności klas (zbiory *Wine Quality*), agresywne przycinanie może prowadzić do utraty istotnych informacji i pogorszenia wyników.

Porównanie implementacji autorskiej z biblioteką scikit-learn potwierdziło poprawność logiczną i matematyczną stworzonych algorytmów. Osiągnięte metryki jakości są zbliżone do wyników referencyjnych, co świadczy o skuteczności przyjętych rozwiązań. Główne różnice objawiły się w czasie obliczeń, wskazując na przewagę wysokopoziomowych optymalizacji stosowanych w bibliotekach produkcyjnych.